MSBA Workshop: Brief Review of Logarithms

Logarithms

- A **logarithm** is an exponent.
- If $y = b^x$ then $\log_b(y) = x$.
- b is called the base.
- A logarithm answers the question: "Compute the power (x) that a base (b) must be raised to in order to return a certain value (y)."
- For example, since $16 = 4^2$ then $\log_4(16) = 2$, and since $16 = 2^4$ then $\log_2(16) = 4$.

1. $\log_{10}(100)$
2. $\log_{10}(1000)$
3. $\log_{10}(10000)$
4. $\log_{10}(10)$
5. $\log_{10}(1)$
6. $\log_8(64)$
7. $\log_2(64)$

- Any number to the power of 0 is 1: $b^0 = 1$.
- So $\log_b(1) = 0$ for any base b.

Logarithms in Python

The libraries `math` and `numpy` include functions for computing logarithms.

```
import math

math.log(100, 10)
```

```
2.0
```

```
math.log(64, 8)
```

```
2.0
```

```
math.log(64, 2)
```

```
6.0
```

The number e

Invest 1 dollar at an annual interest rate of 100%. How much money do you have at the end of 1 year if interest is compounded:

1. Annually
2. Semi-annually
3. Monthly
4. Daily
5. n times in the year
6. Continuously

$$e = \lim_{n \to \infty} \left(1 + \frac{1}{n}\right)^n \approx 2.7183$$

```
import numpy as np

np.exp(1)
```

```
2.718281828459045
```

Exponential function

How much money do you have at the end of 2 years if interest is compounded continuously?

```
import numpy as np

np.exp(2)
```

```
7.38905609893065
```

Invest 1 dollar at an annual interest rate of 10%. How much money do you have at the end of 1 year if interest is compounded:

1. Annually
2. Semi-annually
3. Monthly
4. Daily
5. n times in the year
6. Continuously

$$e^x = \lim_{n \to \infty} \left(1 + \frac{x}{n}\right)^n$$

Natural logarithm

The base-e logarithm is called the **natural logarithm**. In math, it is usually denoted ln.

$$\ln(x) = \log_e(x)$$

However, in data science and in most software packages, Python included, log or `log` refers to "natural log".

```
np.log(np.exp(1))
```

```
1.0
```

```
np.log(np.exp(1) ** 2)
```

```
2.0
```

```
np.log(10)
```

```
2.302585092994046
```

The `numpy` function for base-10 logarithms is `log10`.

```
np.log10(10)
```

```
1.0
```

```
np.log10(100)
```

```
2.0
```

Log Transformations

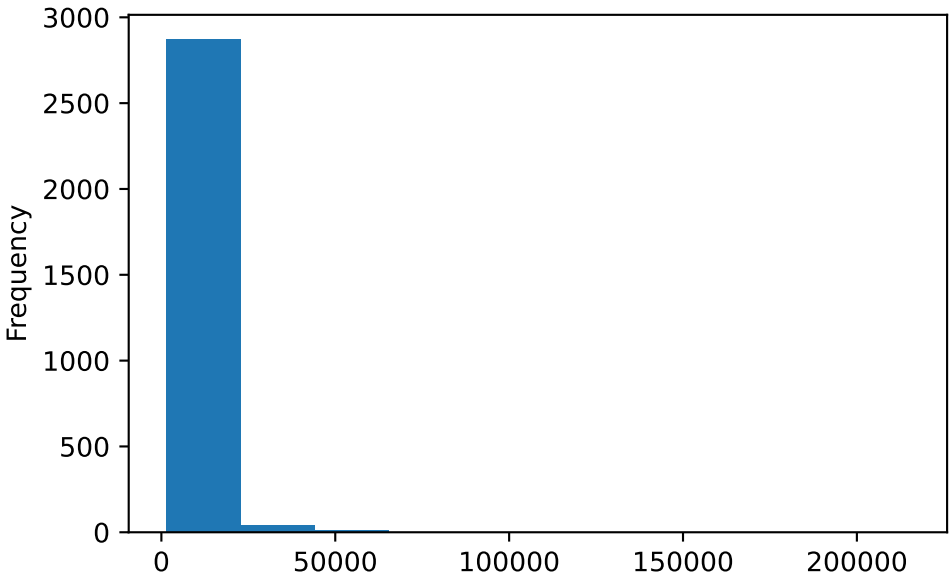
Log transformations of variables are common in data science. Let's look at the lot area variable from the Ames housing data.

```
import pandas as pd
```

```
df_ames = pd.read_csv("https://raw.githubusercontent.com/kevindavisross/data301/main/data/Ame
df_ames = df_ames.sort_values(by = "Lot Area")
df_ames
```

	Order	PID	MS SubClass	MS Zoning	Lot Frontage	Lot Area	Street	Alley	Lot Shape
935	936	909451180	160	RM	24.0	1300	Pave	NaN	Reg
2913	2914	923226180	180	RM	21.0	1470	Pave	NaN	Reg
329	330	923226250	160	RM	21.0	1476	Pave	NaN	Reg
977	978	923228110	180	RM	21.0	1477	Pave	NaN	Reg
1599	1600	923228080	160	RM	21.0	1477	Pave	NaN	Reg
...	...	...	...	...	...	...	...	...	...
2766	2767	906475200	20	RL	62.0	70761	Pave	NaN	IR1
2071	2072	905301050	20	RL	NaN	115149	Pave	NaN	IR2
2115	2116	906426060	50	RL	NaN	159000	Pave	NaN	IR2
1570	1571	916125425	190	RL	NaN	164660	Grvl	NaN	IR1
956	957	916176125	20	RL	150.0	215245	Pave	NaN	IR3

```
df_ames["Lot Area"].plot.hist()
```



There are a few homes with such extreme lot areas that we get virtually no resolution at the lower end of the distribution. Over 95% of the observations are in a single bin of this histogram. In other words, the distribution of this variable is extremely *skewed*.

One way to improve this histogram is to use more bins. But this does not solve the fundamental problem: we need more resolution at the lower end of the scale and less resolution at the higher end. One way to spread out the values at the lower end of a distribution and to compress the values at the higher end is to take the logarithm (provided that the values are all positive). Log transformations are particularly effective at dealing with data that is skewed to the right.

```
df_ames["log(Lot Area)"] = np.log(df_ames["Lot Area"])
df_ames[["Lot Area", "log(Lot Area)"]]
```

	Lot Area	log(Lot Area)
935	1300	7.170120
2913	1470	7.293018
329	1476	7.297091
977	1477	7.297768
1599	1477	7.297768
...	...	...
2766	70761	11.167063
2071	115149	11.653982
2115	159000	11.976659
1570	164660	12.011638

	Lot Area	log(Lot Area)
956	215245	12.279532

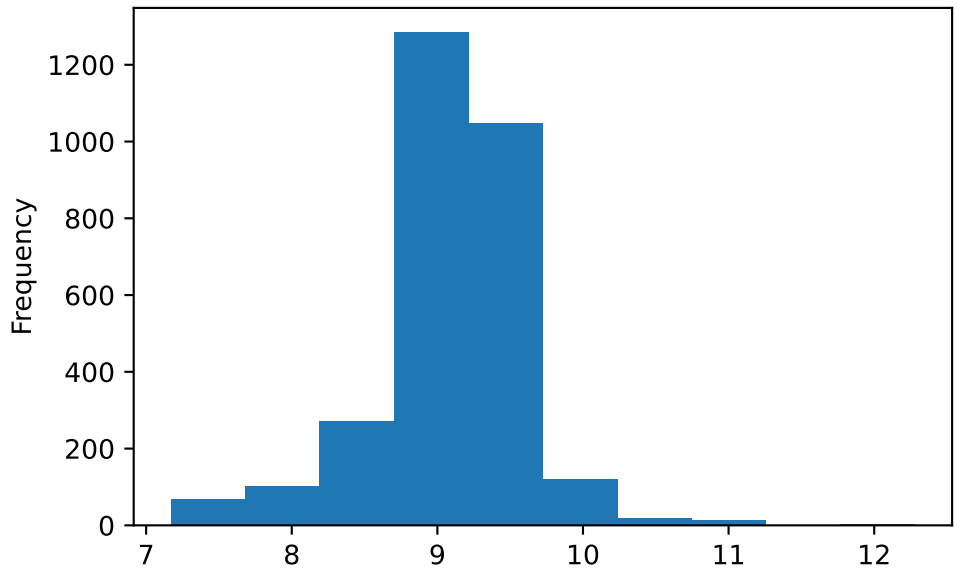
Remember "log" usually refers to "natural log", and that's what we have computed (`np.log`). We could also transform using base-10 log (`np.log10`).

```
df_ames["log10(Lot Area)"] = np.log10(df_ames["Lot Area"])
df_ames[["Lot Area", "log(Lot Area)", "log10(Lot Area)"]]
```

	Lot Area	log(Lot Area)	log10(Lot Area)
935	1300	7.170120	3.113943
2913	1470	7.293018	3.167317
329	1476	7.297091	3.169086
977	1477	7.297768	3.169380
1599	1477	7.297768	3.169380
...	...	...	...
2766	70761	11.167063	4.849794
2071	115149	11.653982	5.061260
2115	159000	11.976659	5.201397
1570	164660	12.011638	5.216588
956	215245	12.279532	5.332933

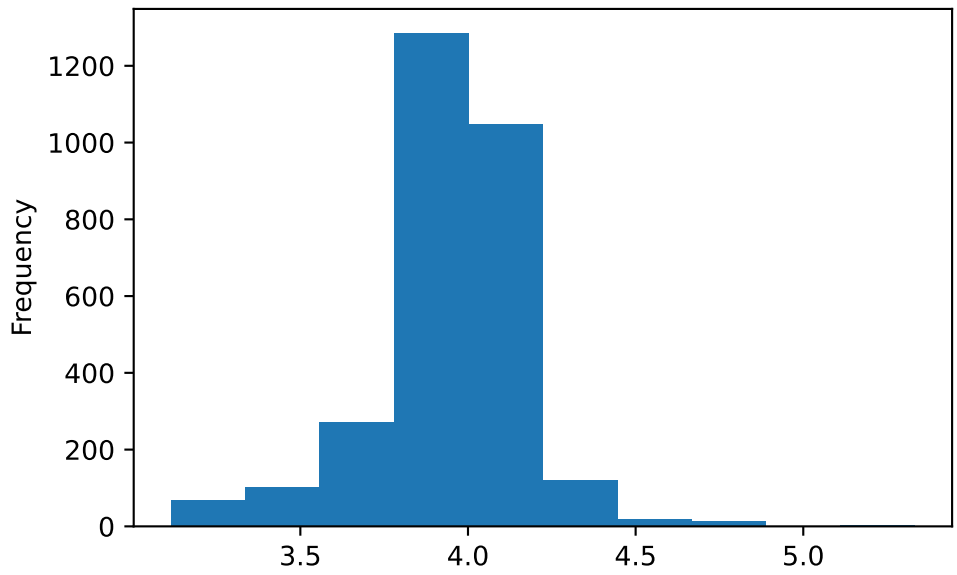
These numbers are not very interpretable on their own, but if we make a histogram of these values, we see that the lower end of the distribution is now more spread out, and the extreme values are not as far out on the logarithmic scale.

```
df_ames["log(Lot Area)"].plot.hist()
```



The effect is the same if we take natural log or base-10 log; the only difference is the scale.

```
df_ames["log10(Lot Area)"].plot.hist()
```



> ⚠ Warning
>
> Do NOT interpret symmetric as "good" and skewed as "bad". The shape of the data is what it is; there is no right or wrong.

- From an analysis perpesctive, the fact that Lot Area was skewed just meant that the histogram—which uses bins of equal width—did not adequately visualize the sample data.
- It is often more natural to view values that span multiple orders of magnitude on a logarithmic/multiplicative scale ("how many *times* bigger?") rather than a linear/absolute scale.
- Taking a log transformation allows us to differentially zoom in to different ranges of the data, achieving better resolution when the distribution is skewed.

Here is a graph from the *NY Times* with GDP per capita on a logarithmic scale.

Log Scale

1. How many *times* bigger is 100 than 10? Is 1000 than 10?
2. How far apart are $\log_{10}(100)$ and $\log_{10}(10)$? $\log_{10}(1000)$ and $\log_{10}(10)$?

- It is often more natural to view values that span multiple orders of magnitude on a logarithmic/multiplicative scale ("how many *times* bigger?") rather than a linear/absolute scale ("how much bigger?").
  - For example, if you think of a number as its leading digit followed by 0s (e.g., think of 4567 as 4000, in the thousands) then the number before the decimal point in the base-10 logarithm is the number of 0s and tells you the order of magnitude (e.g. $\log_{10}(4567) = 3.66$, so there are 3 zeros after the leading digit and 4567 is in the thousands).

Base-10 log	Order of magnitude
1.ish	Tens
2.ish	Hundreds
3.ish	Thousands
4.ish	Tens of thousands
5.ish	Hundreds of thousands
6.ish	Millions

  –

- A one-unit incremement in a base-b log scale corresponds to a b times increment in the original scale.
  - For example, each one-unit increase on a base10 log scale corresponds to a 10 times increase in the original scale
  - So on a base10 log scale, the interval 1 to 2 represents values in the tens, 2 to 3 represents hundreds, 3 to 4 reprents thousands, etc.

Useful properties of logarithms

Roughly, logarithms turn multiplication into addition. The logarithm of a product is the sum of the logarithms.

$$\log_b(xy) = \log_b(x) + \log_b(y)$$

Roughly, logarithms turn exponentiation (powers) into products.

$$\log_b(x^y) = y \log_b(x)$$

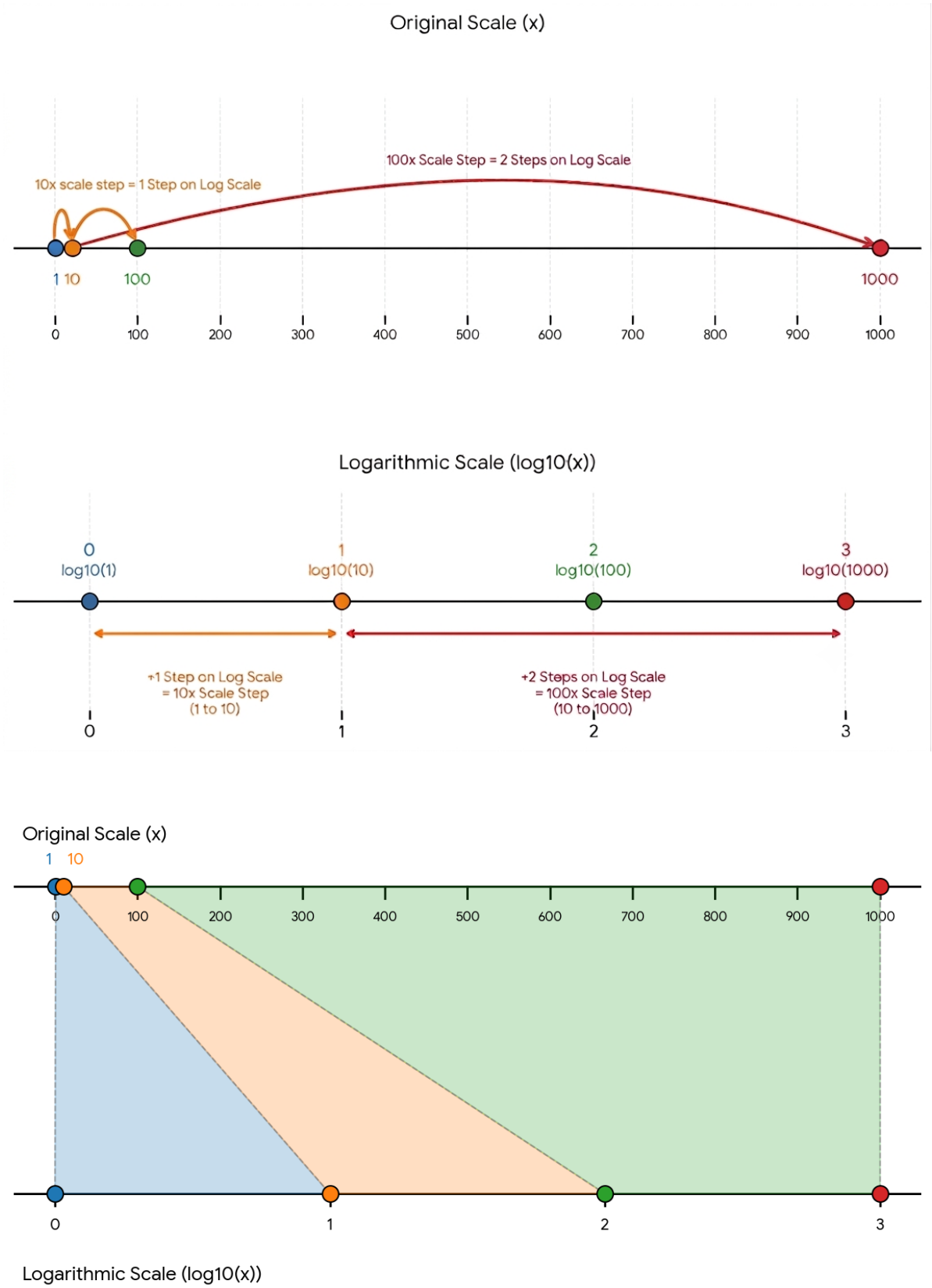


Figure 1: Illustrations of (base 10) log scale (*generated by Gemini*)